UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

CHAPTER 13 LAB MANUAL

Integration, Deployment and Mobile

(Vue 3 frontend + Slim API + Capacitor Android wrap)

FIELD	DETAILS
Course	SCSM2223 — Cross-Platform Application Development
Chapter	13 — Integration, Deployment and Mobile
Lab time	3 hours
Lab assessment	Lab exercise component
Software	Laragon, Composer, Node 18+, npm, Sublime Text/VS Code, Android Studio (optional)
Builds on	Ch12_BooksAPI_Solution (the hardened backend)
Companion	Reference solution: Ch13_FullStack_Solution/

1. Lab Objectives

By the end of this lab you will be able to:

1. Scaffold a Vue 3 + Vite project alongside an existing Slim+JWT API.
2. Configure environment variables so the same code targets local dev and production.
3. Wire one Axios client with a JWT interceptor and an auto-logout on 401.
4. Build a Pinia store for authentication state and persistence.
5. Implement Login, Register, BookList, BookForm, and Profile views.
6. Produce a production build (npm run build) and serve it locally.
7. Wrap the built web app with Capacitor and run it on Android.
8. Outline a deployment plan covering frontend, backend, database, and mobile.

2. Prerequisites

Verify each item before starting.

#	Tool	Verify by running…	Expected
1	Laragon	Open Laragon, click Start All	Apache + MySQL green
2	PHP 8.1+	php -v	PHP 8.1.x
3	Node.js 18+	node -v	v18 or v20
4	npm	npm -v	9.x or 10.x
5	Ch12 backend running	Visit http://localhost:8000/	JSON health response
6	Android Studio (optional)	Open Android Studio	Android SDK installed

TIP If you skip Android Studio, you can still complete every step except the final Capacitor run. The web app will still be fully working.

3. What We Will Build

A Vue 3 single-page app called "UTM Books" that wraps the Chapter 12 API. By the end of the lab the same source will run as a website on http://localhost:5173 and as an Android app via Capacitor.

Part A — Scaffold the Frontend

Step 1 — Create the Vite project

From a folder of your choice (NOT inside C:\laragon\www), run:

```
npm create vue@latest books-frontend
  ✓ Add Pinia                     …  Yes
  ✓ Add Vue Router                 …  Yes
```

```
  (decline ESLint / Prettier / etc. for the lab)

cd books-frontend
npm install
npm install axios

npm run dev          # opens http://localhost:5173
```

Step 2 — Add environment files

Create two files in the project ROOT (next to package.json). These tell Vite which API URL to embed at build time.

```
# .env.development
VITE_API_BASE_URL=http://localhost:8000

# .env.production
VITE_API_BASE_URL=https://api.books.utm.my   # replace with your URL when deploying
```

WHY Vite inlines variables prefixed with VITE_ at BUILD time. Anything else stays server-side. Never put real secrets here — the bundle is shipped to browsers.

Part B — Auth Store & Axios Client

Step 3 — src/stores/auth.js

Replace the auto-generated counter store with this auth store. It persists `token` and `user` in `localStorage` and exposes the getters used everywhere in the UI.

```
import { defineStore } from 'pinia';
import axios from 'axios';

const baseURL = import.meta.env.VITE_API_BASE_URL;

export const useAuth = defineStore('auth', {
  state: () => ({
    token: localStorage.getItem('token') || null,
    user:  JSON.parse(localStorage.getItem('user') || 'null'),
  }),
  getters: {
    isAuthenticated: (s) => !!s.token,
    isAdmin:         (s) => s.user?.role === 'admin',
  },
  actions: {
    async login(email, password) {
      const { data } = await axios.post(`${baseURL}/auth/login`, { email, password });
      this.token = data.access_token;
      this.user  = data.user;
      localStorage.setItem('token', this.token);
      localStorage.setItem('user',  JSON.stringify(this.user));
    },
    async register(name, email, password) {
      await axios.post(`${baseURL}/auth/register`, { name, email, password });
```

```
      await this.login(email, password);
    },
    logout() {
      this.token = null;
      this.user  = null;
      localStorage.removeItem('token');
      localStorage.removeItem('user');
    },
  },
});
```

Step 4 — src/api/client.js

One Axios instance for the whole app. The request interceptor attaches the JWT; the response interceptor logs out on 401.

```
import axios from 'axios';
import { useAuth } from '../stores/auth';

const api = axios.create({
  baseURL: import.meta.env.VITE_API_BASE_URL,
  timeout: 10_000,
});

api.interceptors.request.use((cfg) => {
  const auth = useAuth();
  if (auth.token) cfg.headers.Authorization = `Bearer ${auth.token}`;
  return cfg;
});

api.interceptors.response.use(
  (res) => res,
  (err) => {
    if (err.response?.status === 401) useAuth().logout();
    return Promise.reject(err);
  }
);

export default api;
```

Part C — Router & Views

Step 5 — src/router/index.js

```
import { createRouter, createWebHistory } from 'vue-router';
import { useAuth } from '../stores/auth';

const router = createRouter({
  history: createWebHistory(),
  routes: [
    { path: '/',         name: 'books',    component: () => import('../views/BookList.vue') },
    { path: '/login',    name: 'login',    component: () => import('../views/Login.vue') },
    { path: '/register', name: 'register', component: () => import('../views/Register.vue') },
    { path: '/profile',  name: 'profile',  component: () => import('../views/Profile.vue'),
```

```
      meta: { requiresAuth: true } },
  ],
});

router.beforeEach((to) => {
  const auth = useAuth();
  if (to.meta.requiresAuth && !auth.isAuthenticated) {
    return { name: 'login', query: { redirect: to.fullPath } };
  }
});

export default router;
```

Step 6 — Login.vue (essentials)

```
<script setup>
import { ref } from 'vue';
import { useRouter } from 'vue-router';
import { useAuth } from '../stores/auth';

const auth = useAuth();
const router = useRouter();
const email = ref('member@books.test');
const password = ref('password');
const error = ref('');

async function submit() {
  try {
    await auth.login(email.value, password.value);
    router.push('/');
  } catch (e) {
    error.value = e.response?.data?.error || e.message;
  }
}
</script>

<template>
  <p v-if="error" style="color:red">{{ error }}</p>
  <input v-model="email" placeholder="Email" />
  <input v-model="password" type="password" placeholder="Password" />
  <button @click="submit">Sign in</button>
</template>
```

Step 7 — BookList.vue (essentials)

```
<script setup>
import { ref, onMounted } from 'vue';
import api from '../api/client';

const books = ref([]);
const q = ref('');

async function load() {
  const { data } = await api.get('/api/books', { params: { q: q.value || undefined } });
  books.value = data.data;
```

```
}
onMounted(load);
</script>

<template>
  <input v-model="q" @keyup.enter="load" placeholder="Search title or author" />
  <button @click="load">Search</button>
  <ul>
    <li v-for="b in books" :key="b.id">
      <strong>{{ b.title }}</strong> — {{ b.author }} ({{ b.year }})
    </li>
  </ul>
</template>
```

(See the reference solution for full versions including Register, Profile, and the BookForm component with create/edit and ownership-aware actions.)

Part D — End-to-end Test (Dev)

With the Ch12 backend running on http://localhost:8000 and the frontend on http://localhost:5173, walk through:

9. Open http://localhost:5173/login and sign in as member@books.test / password.
10. Verify the JWT is stored under localStorage.token in DevTools.
11. Navigate to / and search for 'clean'. Books appear from the API.
12. Click + New Book, fill in title/author/year, save → 201 Created.
13. As a member, edit YOUR new book → 200 OK. Edit a book you don't own → 403.
14. Logout, login as admin@books.test / password, delete any book → 200.
15. Open Profile → /auth/me returns your user payload.

CHECKPOINT If the API call from the browser fails with a CORS error, make sure CORS_ALLOWED_ORIGINS in the backend .env contains http://localhost:5173 and the backend has been restarted.

Part E — Production Build

Step 8 — Confirm .env.production

Update .env.production to point at the URL where you'll deploy the API. For local production-mode testing you can also point it back at localhost:8000.

Step 9 — Build and preview

```
npm run build         # outputs ./dist with hashed asset filenames
npm run preview       # serves dist/ on http://localhost:4173
```

Open http://localhost:4173 and verify the app still works. This is the exact set of files you'll deploy to a static host.

Step 10 — Inspect the bundle

```
dir dist
  index.html
  assets\
    index-XXXX.js
    index-XXXX.css
```

WHY HASHED FILENAMES	Each build gets a unique hash so caches refresh automatically when you redeploy.

Part F — Deploy (overview)

Pick a free / cheap host. The reference solution includes a step-by-step DEPLOYMENT.md; below is the short version.

Frontend → Vercel (one-time)

```
npm i -g vercel
vercel login
vercel --prod          # uploads dist/, returns an HTTPS URL

# Set the env var in the Vercel dashboard:
#   VITE_API_BASE_URL = https://your-api-url
```

Backend → Render Web Service

Push the Ch12 backend repo to GitHub, then create a Web Service on Render. Required env vars: DB_*, JWT_SECRET, JWT_TTL, JWT_ISSUER, CORS_ALLOWED_ORIGINS (include the frontend URL), APP_DEBUG=false.

Database → PlanetScale or shared hosting MySQL

Apply sql/schema.sql once. Set the connection details in the backend's environment variables.

Final checks

1. Frontend can reach the deployed API (no CORS errors in browser console).
2. HTTPS works and the cert is valid.
3. Login + protected calls work end-to-end on the deployed URLs.

Part G — Capacitor: Android Wrap

Step 11 — Add Capacitor

```
npm install @capacitor/core @capacitor/cli
npx cap init books-app com.utm.books --web-dir=dist

npm install @capacitor/android
npx cap add android        # creates the android/ folder
```

Step 12 — Sync the built web app

```
# 1. Build the web app FIRST — Capacitor copies dist/ into android/
npm run build
```

```
npx cap sync android

# 2. Open Android Studio (must be installed)
npx cap open android
```

Step 13 — Run on emulator / device

4. Wait for Gradle sync to finish in Android Studio (a couple of minutes the first time).
5. Pick an emulator under Device Manager, OR connect a phone with USB debugging enabled.
6. Click ▶ Run. The app installs and launches.
7. Sign in with a seeded user, browse books, create a book — everything works in the WebView.

PRODUCTION TIP	Because the app is just a WebView, it talks to your DEPLOYED API, not localhost. Make sure VITE_API_BASE_URL was the production URL when you ran npm run build before npx cap sync.

Step 14 — (Optional) Generate a release APK

```
# 1. Generate a keystore (do this ONCE, store it safely):
keytool -genkey -v -keystore release.keystore -alias books -keyalg RSA -keysize 2048 -validity
10000

# 2. In Android Studio:
#    Build → Generate Signed Bundle / APK → APK → Release
#    Pick your keystore. Output: app-release.apk

# 3. Side-load on a phone, or upload to Google Play Console.
```

Part H — Self-Check & Extension Tasks

8. Confirm a non-logged-in user is redirected from /profile to /login.
9. Confirm /api/books?q=clean returns the expected book in the UI.
10. Confirm token persistence — refresh the browser, you stay logged in.
11. Confirm 401 from the API auto-logs the user out (delete localStorage.token mid-session and refetch).
12. Confirm a member cannot edit a book they don't own (Edit button hidden).
13. Build with .env.production and verify the built bundle uses the production URL (Network tab).
14. BONUS — Add a Capacitor Camera plugin and let users add a cover image to a book.
15. BONUS — Add Sentry (or simple error logging) to capture frontend errors in production.
16. BONUS — Add an /api/health endpoint to the backend and a UI badge showing API status.

Appendix — Common Errors & Troubleshooting

Symptom	Likely cause	Fix
CORS error in DevTools console	Backend doesn't list the frontend origin	Add http://localhost:5173 (and the deployed URL) to

Symptom	Likely cause	Fix
		CORS_ALLOWED_ORIGINS, restart the backend.
401 on every protected call	Token not attached	Check axios interceptor uses useAuth().token; check the request headers in DevTools.
Network error / no response	Wrong VITE_API_BASE_URL	Verify the URL with curl. Rebuild after changing .env.
npm run build fails with import errors	Missing component file	Make sure every router import path is correct and case-sensitive.
Capacitor sync fails	android/ not added	Run npx cap add android first.
WebView opens but can't reach API	App on phone can't reach localhost	Use a deployed API URL, OR start the backend on the LAN IP and update VITE_API_BASE_URL.
HTTPS certificate warning on phone	Self-signed / dev cert	Use a real cert on the deployed API. Capacitor sets androidScheme: https for https-only WebViews.

End of Lab Manual.

End of SCSM2223 Cross-Platform Application Development.